Name: - Shubham Das **S. A. P I. D: - 590033500**
Date:- 30/08/2026 **Subject: - Programming in C**

Question 1: - A square field has ‘s’ meters. A square pond of side ‘p’ meters is inside it. Write a C programme to find the remaining land area.

Part 1: - Thorough understanding of the problem statement
Input: - ‘s’: Side of the square field (in meters)
‘p’: Side of the square pond (in meters)

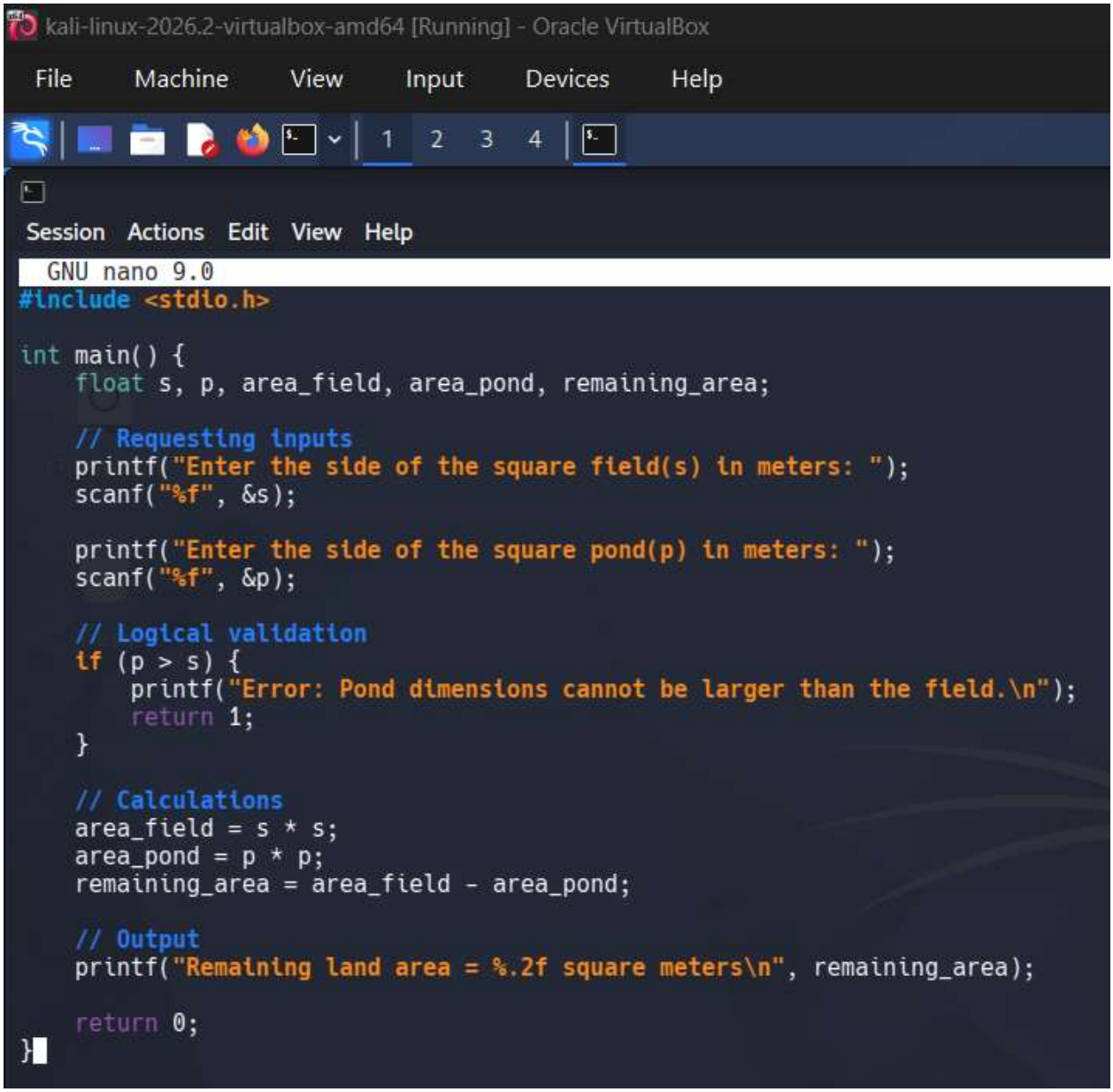
Output: - Remaining area of the land (in square meters)

Formula: - Area of the square field = s * s
Area of the square pond = p * p
Remaining Area = (s * s) - (p * p)

Part 2: - Writing, compiling and evaluating the algorithm

1. Start
2. Declare all the floating-point variables: ‘s’, ‘p’, ‘area_field’, ‘area_pond’, ‘remaining_area’.
3. Display the following particular message to the user: “Enter the side of the square field (s): ”
4. Read input into variable ‘s’.
5. Display the following particular message to the user: “Enter the side of the square pond (p): ”
6. Read input into variable ‘p’.
7. Check if p > s. If true, print an error (pond cannot be larger than the field) and terminate.
8. Calculate “area_field = s * s”.
9. Calculate “area_pond = p * p”.
10. Calculate “remaining_area = area_field - area_pond”.
11. Print the value of the “remaining_area”.
12. Stop

Part 3:- Writing the C programme

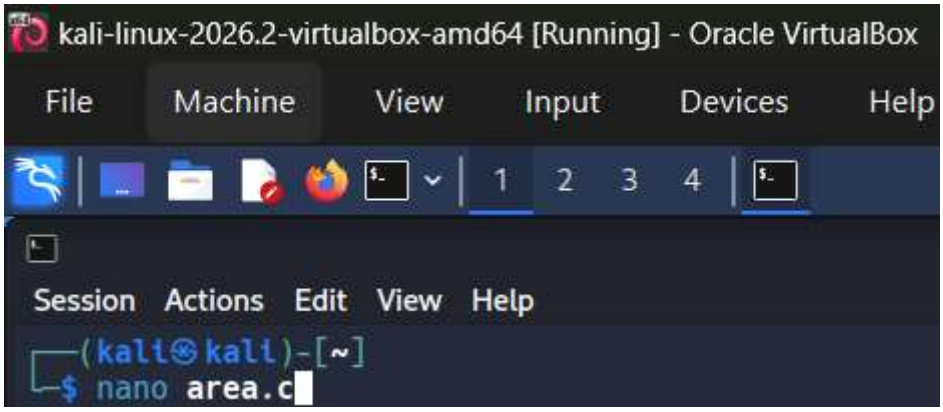


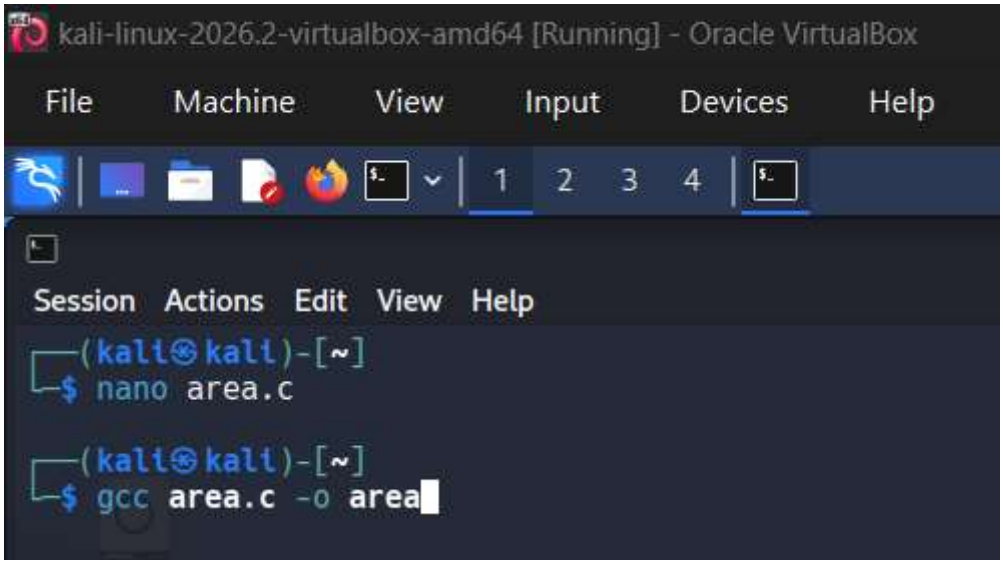
Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

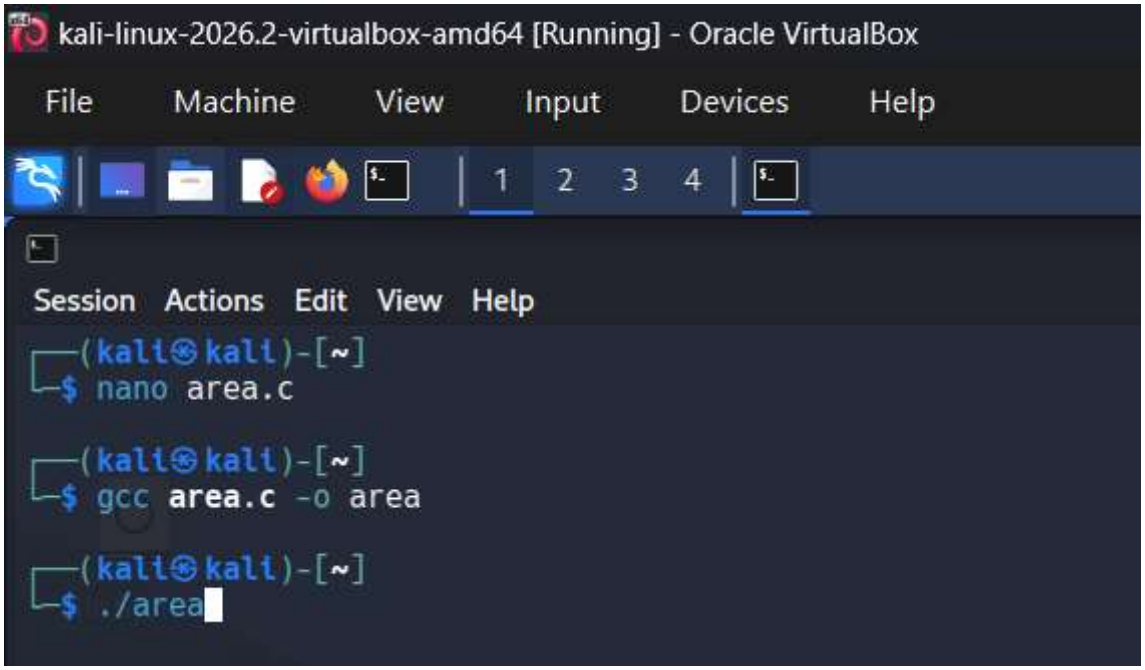
Step 1: - Open the Terminal
Step 2: - Use the ‘nano’ text editor to create a new file named ‘area.c’:
Step 3: - Compile the C code into the Terminal windows.
Step 4: - Save the file by pressing “Ctrl + O”, then press ‘Enter’ to confirm the file name.
Step 5: - Exit the editor by pressing “Ctrl + X”.
Step 6: - Use the GNU C Compiler (GCC) to compile your code into an executable file named ‘area’.
(Note: If you do not see any output after running this command, it means the code compiled successfully with no errors.)

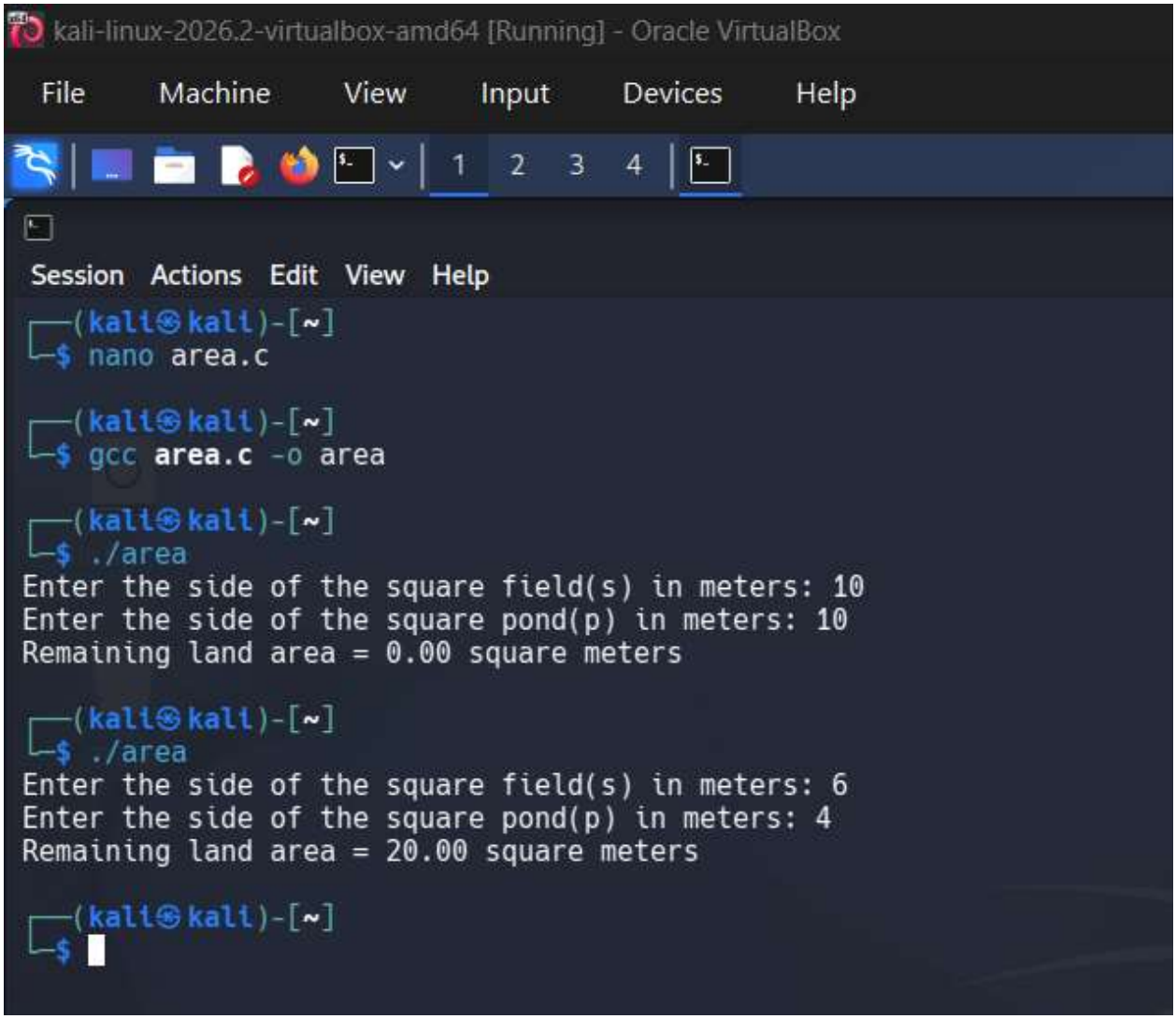
Step 7:- Execute the compiled program by typing **./area'.**

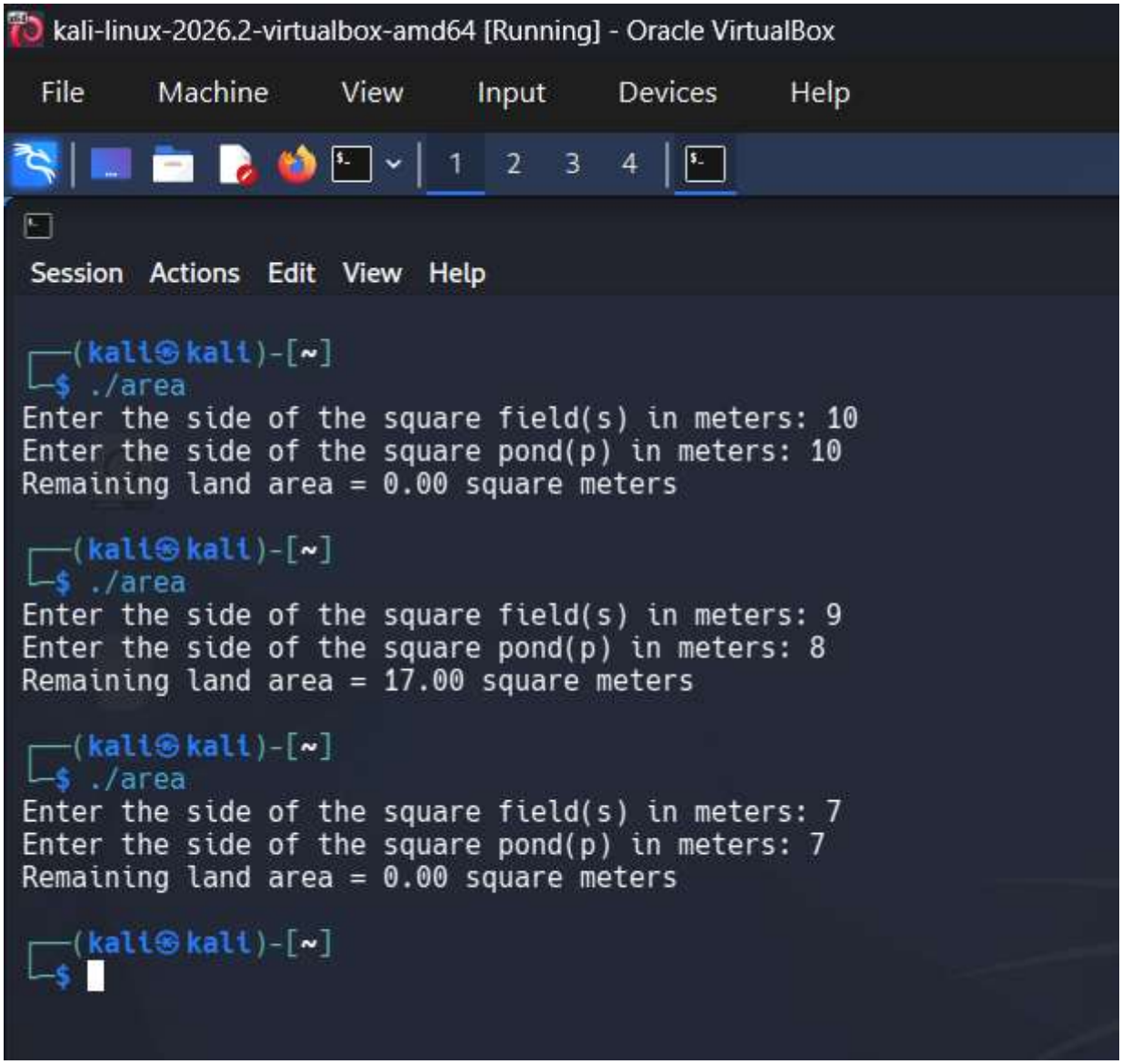
The program will start, and you will see the prompt asking you to enter the dimensions for the field and pond.

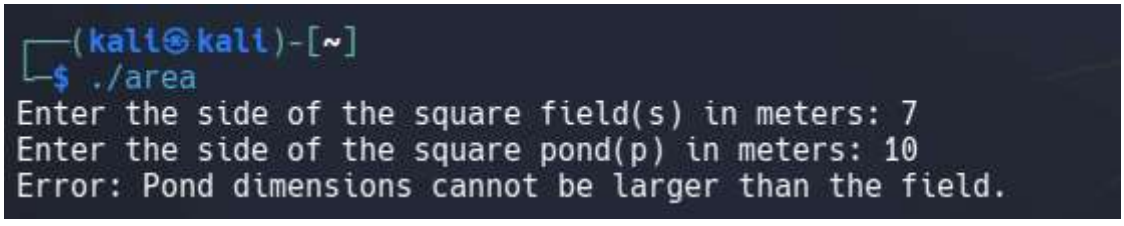





kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
Session Actions Edit View Help
(kali㉿kali)-[~]
$ nano area.c
(kali㉿kali)-[~]
$ gcc area.c -o area
(kali㉿kali)-[~]
$ ./area


kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
Session Actions Edit View Help
(kali㉿kali)-[~]
$ nano area.c
(kali㉿kali)-[~]
$ gcc area.c -o area
(kali㉿kali)-[~]
$ ./area
Enter the side of the square field(s) in meters: 10
Enter the side of the square pond(p) in meters: 10
Remaining land area = 0.00 square meters
(kali㉿kali)-[~]
$ ./area
Enter the side of the square field(s) in meters: 6
Enter the side of the square pond(p) in meters: 4
Remaining land area = 20.00 square meters
(kali㉿kali)-[~]
$





Part 5: - Compiling, executing and evaluating against all the test cases.
Upon compiling and executing the code with the inputs from **Part 3**, we will check whether or not the calculated outputs match the expected outputs.
Test Case 1 (6, 4): Evaluates to 36 - 16 = 20. Actual output is 20.00. **(Pass)**
Test Case 2 (10, 10): Evaluates to 100 - 100 = 0. Actual Output is 0.00. **(Pass)**

Test Case 3 (9, 8): Evaluates to 81 - 64 = 17. Actual Output is 17.00. **(Pass)**

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	**S = 6, P = 4**	**20**	**20**	**PASS**
2	**S = 10, P =10**	**0**	**0**	**PASS**
3	**S = 9, P = 8**	**17**	**17**	**PASS**

Part 6: - Evaluation: The algorithm runs in constant time and uses constant space. It correctly applies the geometric formulas and includes a basic logical check (the pond cannot be bigger than the field) to ensure real-world validity.

Question 2: - Write a C programme to find the painting cost and border length of a trapezium-shaped board when all sides, height, and painting rate per square meter are given.

Part 1: - Thorough understanding of the problem statement
Input: - Parallel sides: ‘a’ and ‘b’
Non-parallel sides: ‘c’ and ‘d’
Height of the trapezium: ‘h’
Painting rate per square meter: ‘rate’

Output: - Border Length (Perimeter)
Total painting cost

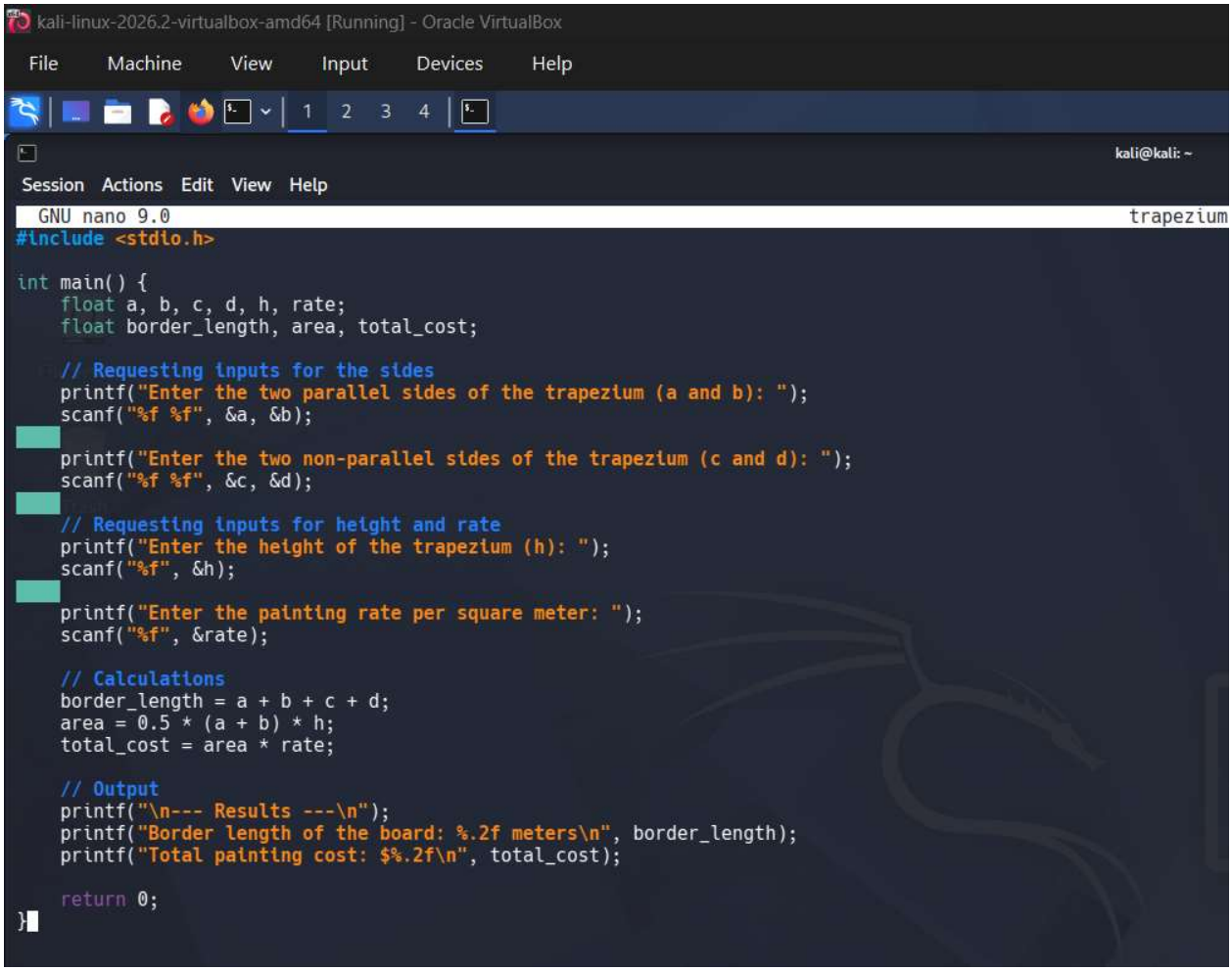
Formula: - Border Length = a+b+c+d
Area of Trapezium = 0.5 * (a+b) * h
Painting Cost = Area * rate

Part 2: - Writing, compiling and evaluating the algorithm

1. Start
2. Declare all the floating-point variables: ‘a’, ‘b’, ‘c’, ‘d’, ‘h’, ‘rate’, ‘border_length’, ‘area’, ‘total_cost’.
3. Display a prompt and read inputs for ‘a’, ‘b’, ‘c’, ‘d’ (the four sides).
4. Display a prompt and read input for ‘h’ (height).

5. Display a prompt and read input for “rate” (painting rate).
6. Calculate “border_length = a+ b+ c + d”.
7. Calculate “area = 0.5 * (a + b) * h”.
8. Calculate “total_cost = area * rate”.
9. Print “border_length”.
10. Print “total_cost”.
11. Stop

Part 3:- Writing the C programme

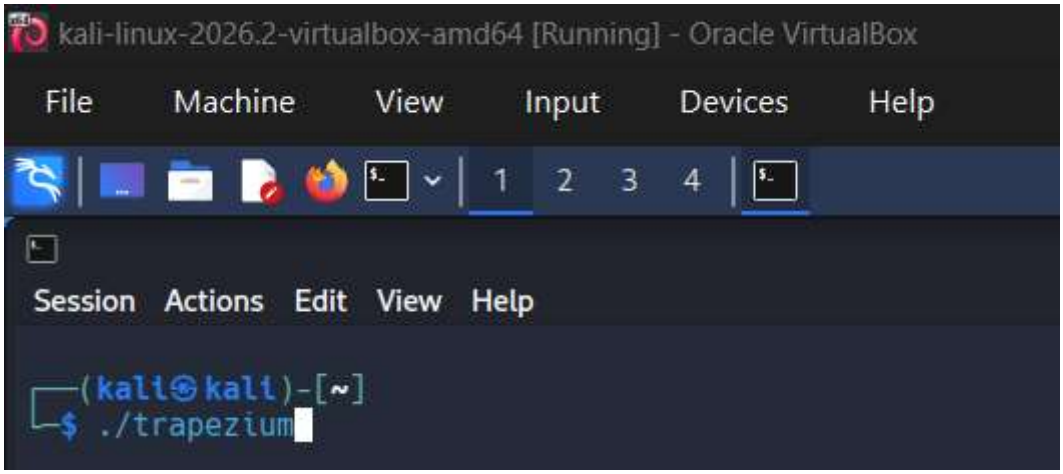


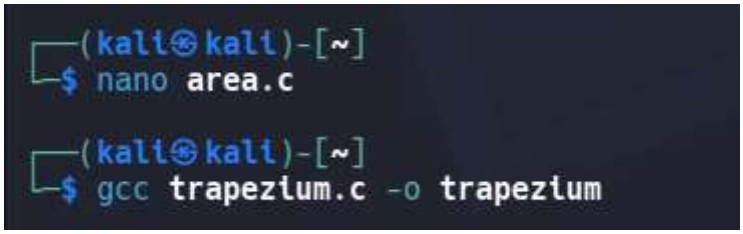
Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

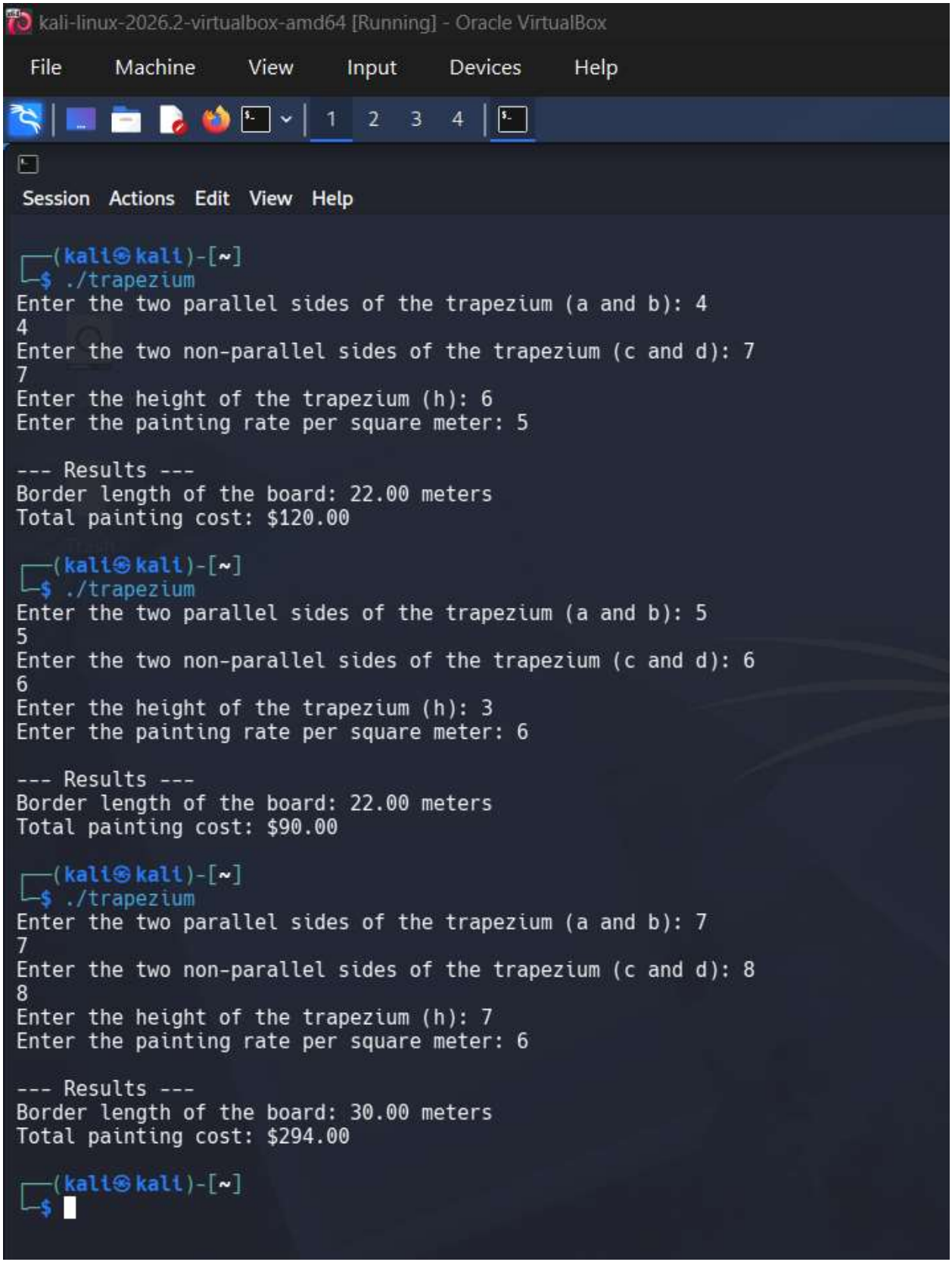
Step 1: - Open the Terminal
Step 2: - Use the ‘nano’ text editor to create a new file named ‘trapezium.c’:
Step 3: - Compile the C code into the Terminal windows.

Step 4: - Save the file by pressing “Ctrl + O”, then press ‘Enter’ to confirm the file name.
Step 5: - Exit the editor by pressing “Ctrl + X”.
Step 6: - Use the GNU C Compiler (GCC) to compile your code into an executable file named ‘trapezium’.
(Note: If the terminal just drops down to a new empty prompt line, your code compiled successfully with no errors. .)
Step 7:- Execute the compiled program by typing ‘./trapezium’.

The program will start, and you will see the prompt asking you to enter the two parallel and non-parallel sides along with height and painting rate (per square meter) of the trapezium.







Part 5: - Compiling, executing and evaluating against all the test cases.

Upon compiling and executing the code with the inputs from **Part 3**, we will check whether or not the calculated outputs match the expected outputs.
Test Case 1 (4, 4, 7, 7, 6, 5): Evaluates to Border = 4+4+7+7 = 22 and Area = 0.5 * 8 * 6 = 24, making Total Cost = 24 * 5 = 120. Actual outputs are 22.00 meters and $120.00. **(Pass)**
Test Case 2 (5, 5, 6, 6, 3, 6): Evaluates to Border = 5+5+6+6 = 22 and Area = 0.5 * 10 * 3 = 15, making Total Cost = 15 * 6 = 90. Actual outputs are 22.00 meters and $90.00. **(Pass)**
Test Case 3 (7, 7, 8, 8, 7, 6): Evaluates to Border = 7+7+8+8 = 30 and Area = 0.5 * 14 * 7 = 49, making Total Cost = 49 * 6 = 294. Actual outputs are 30.00 meters and $294.00. **(Pass)**

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	**4-4, 7-7, 6, 5**	**22 meters, $120**	**22 meters, $120**	**PASS**
2	**5-5, 6-6, 3, 6**	**22 meters, $90**	**22 meters, $ 90**	**PASS**
3	**7-7, 8-8, 7, 6**	**30 meters, $294**	**30 meters, $294**	**PASS**

Part 6: - Evaluation: The algorithm efficiently calculates both required outputs sequentially.Operations are purely arithmetic and will handle floating-point values accurately.